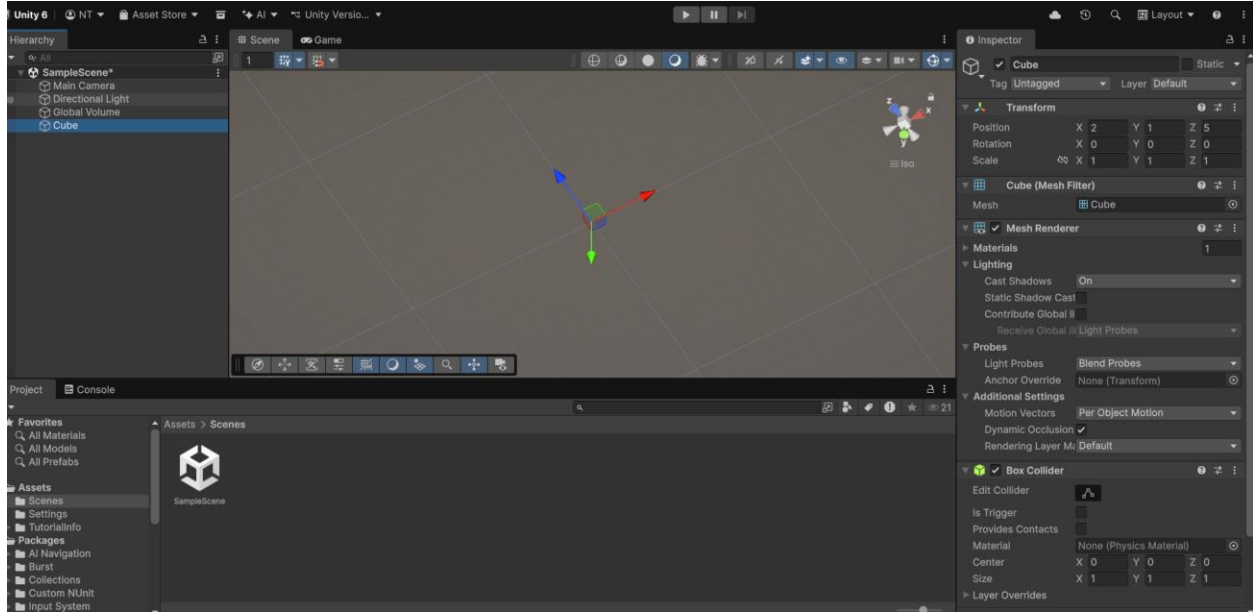


Báo Cáo

PHẦN A – COORDINATE SYSTEM & WORLD SPACE

A1+A2



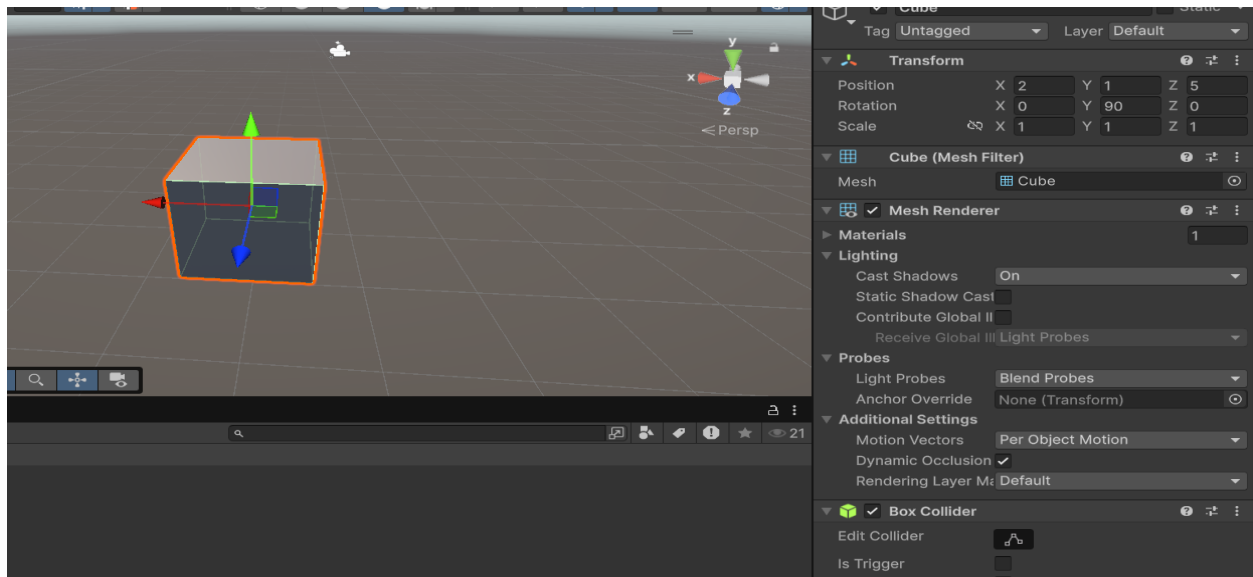
A3

- Trục Y hướng lên trên
- Trục Z dương (+Z) hướng về phía trước, cùng hướng với Camera mặc định trong Unity.

PHẦN B – LEFT-HANDED COORDINATE SYSTEM

B1. Xoay Cube với Rotation:

Y = 90

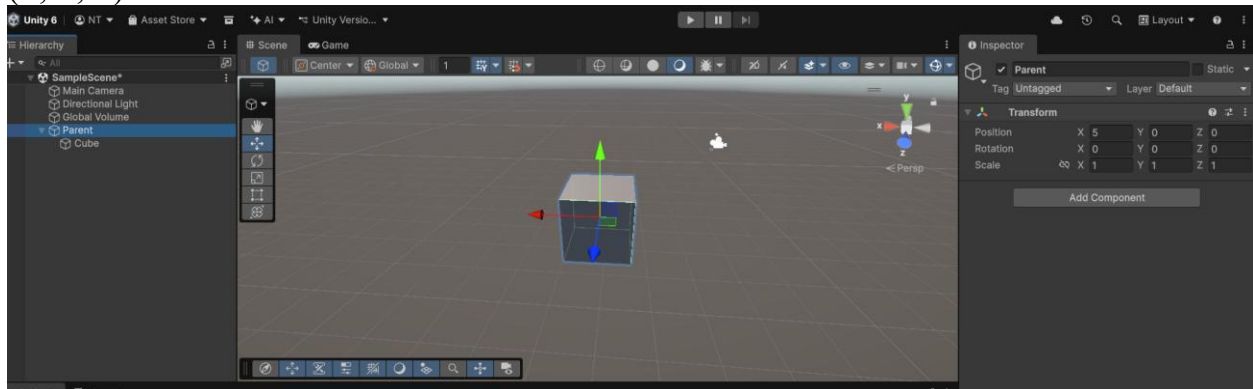


B2

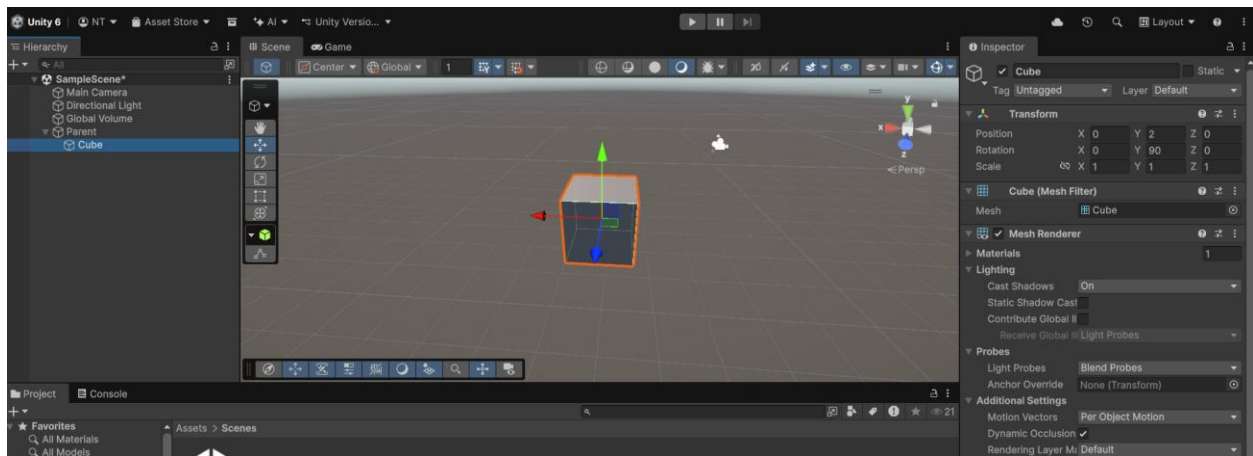
__ Khi xoay Cube 90 độ quanh trục Y, Cube quay theo chiều kim đồng hồ. Điều này cho thấy Unity sử dụng hệ tọa độ Left-Handed, trong đó trục Z dương hướng về phía trước và chiều quay dương tuân theo quy tắc bàn tay trái.

PHẦN C – LOCAL SPACE VÀ WORLD SPACE (25%)

C1. Tạo một Empty GameObject tên là “Parent” tại vị trí: (5, 0, 0)



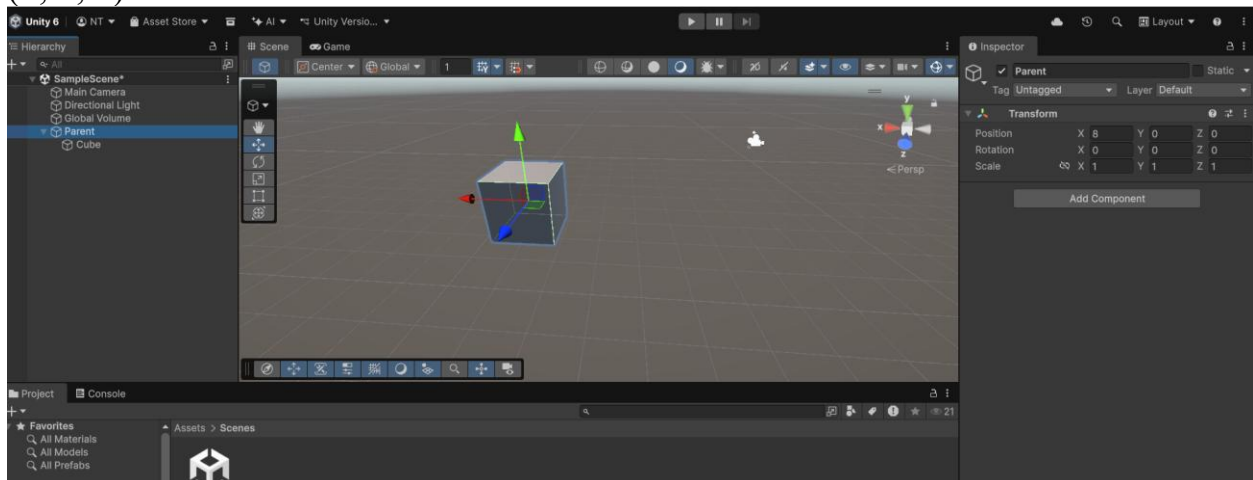
C2. Đặt Cube làm con của Parent và thiết lập: Local Position của Cube = (0, 2, 0)



C3

- Local Position của Cube: (0, 2, 0)
- World Position của Cube: (5, 2, 0)

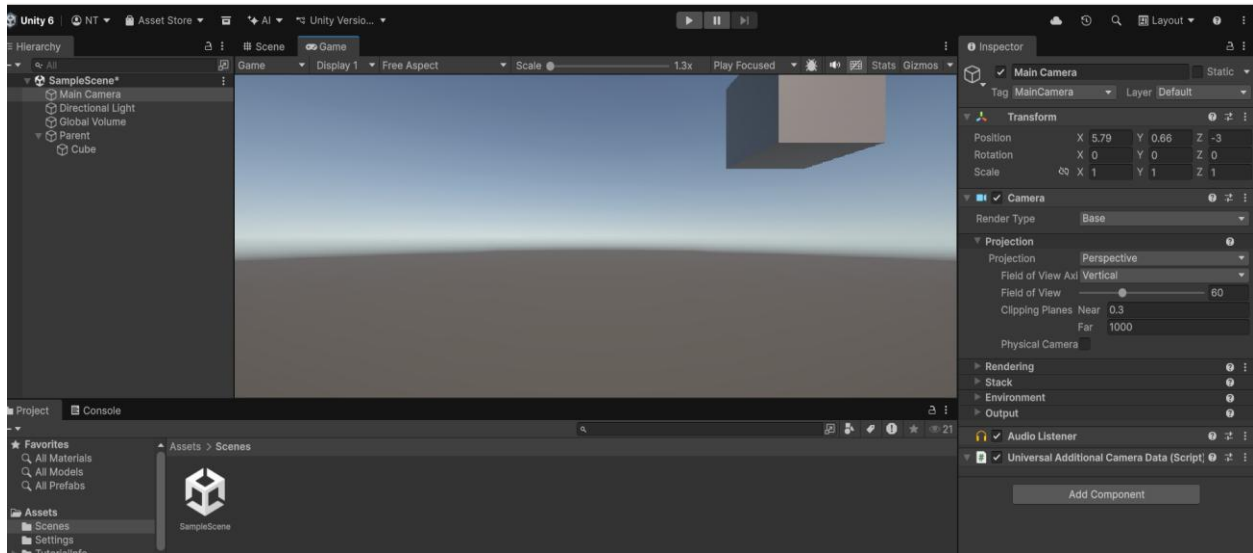
C4. Di chuyển Parent sang vị trí: (8, 0, 0)



- _ Local Position của Cube không thay đổi
- _ World Position của Cube trở thành: (8, 2, 0)

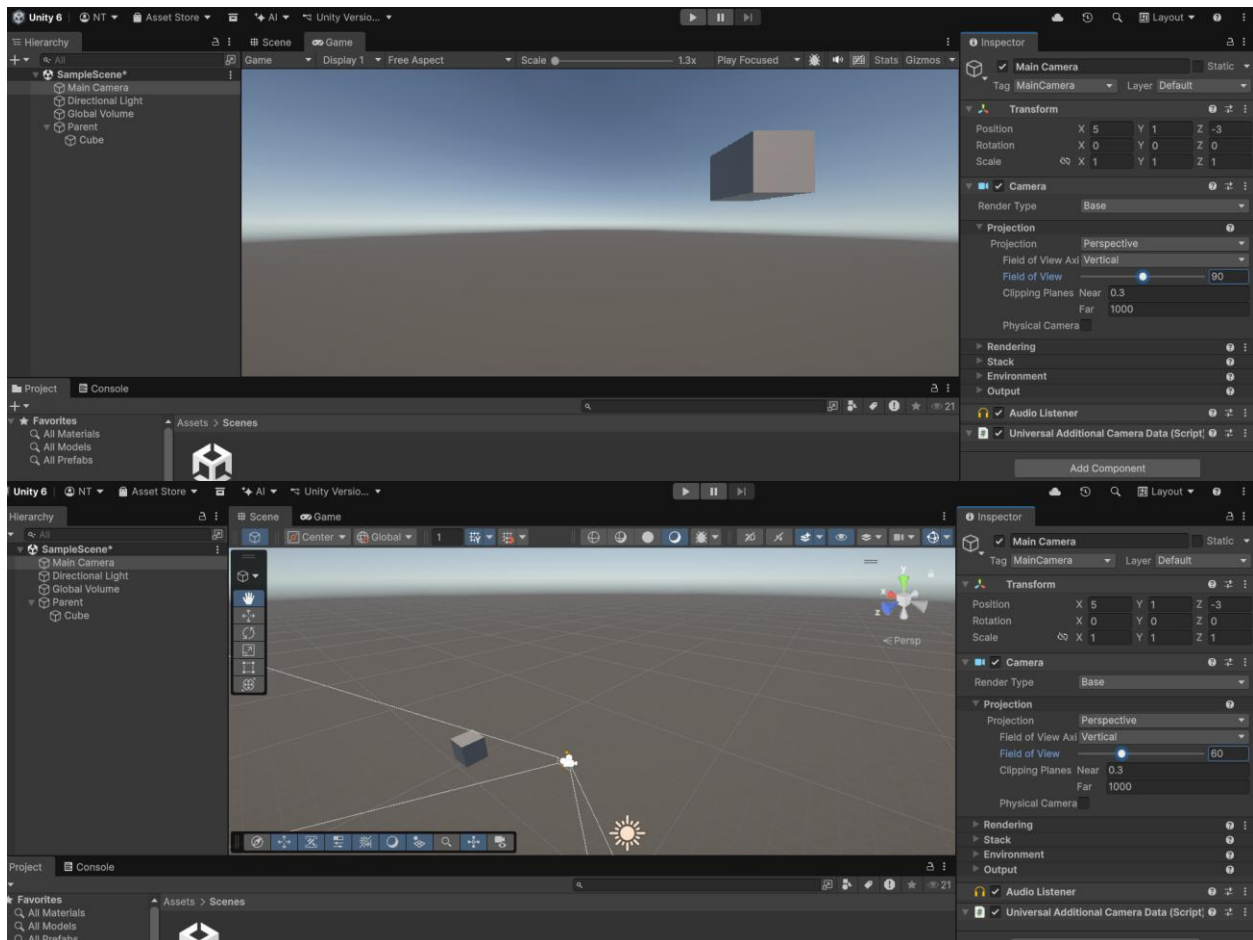
PHẦN D – GRAPHICS PIPELINE (20%)

D1. Di chuyển Camera dọc trục Z từ -10 đến -3

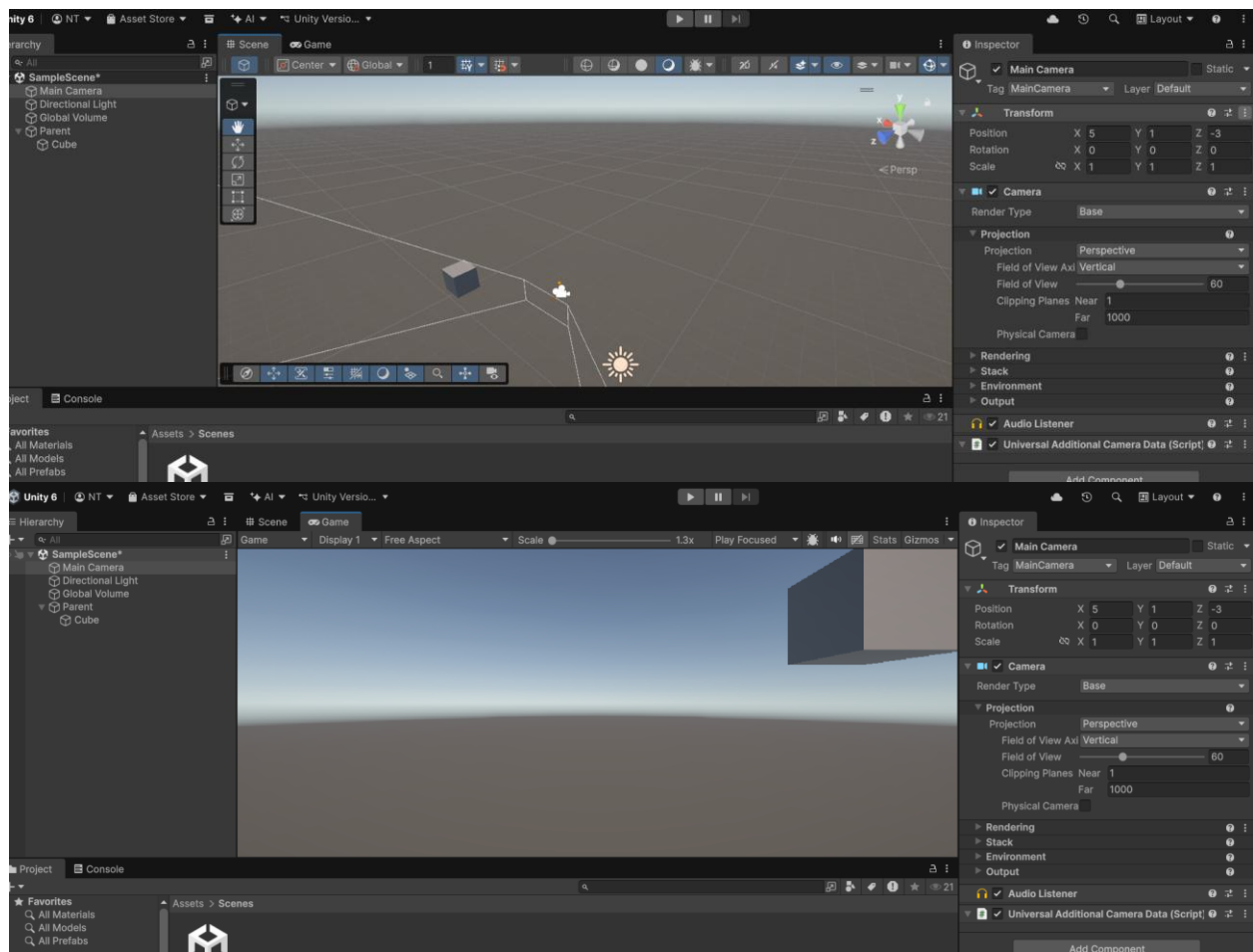


D2. Thay đổi các thông số của Camera:

- Field of View 60 đến 90



- Near Clip Plane: 0.3 đến 1



- Vì sao object trông to hoặc nhỏ hơn dù không đổi vị trí?

Object có thể trông to hoặc nhỏ hơn mặc dù không thay đổi vị trí vì Camera di chuyển gần hoặc xa so với object, hoặc do thay đổi thông số Field of View. Camera là thành phần thực hiện phép chiếu từ không gian 3D sang không gian 2D, nên khi khoảng cách Camera–object hoặc góc nhìn thay đổi, kích thước hiển thị của object trên màn hình cũng thay đổi theo.

- Vì sao object có thể biến mất khỏi màn hình?

Object có thể biến mất khỏi màn hình khi nó nằm ngoài vùng nhìn của Camera, cụ thể là ngoài Near Clip Plane, Far Clip Plane hoặc ngoài View Frustum. Trong quá trình xử lý của Graphics Pipeline, các object nằm ngoài vùng hiển thị sẽ bị loại bỏ (clipping) và không được vẽ lên màn hình.

PHẦN E – SCREEN SPACE (20%)

E1. Tạo script WorldToScreen.cs với nội dung:

```
using UnityEngine;

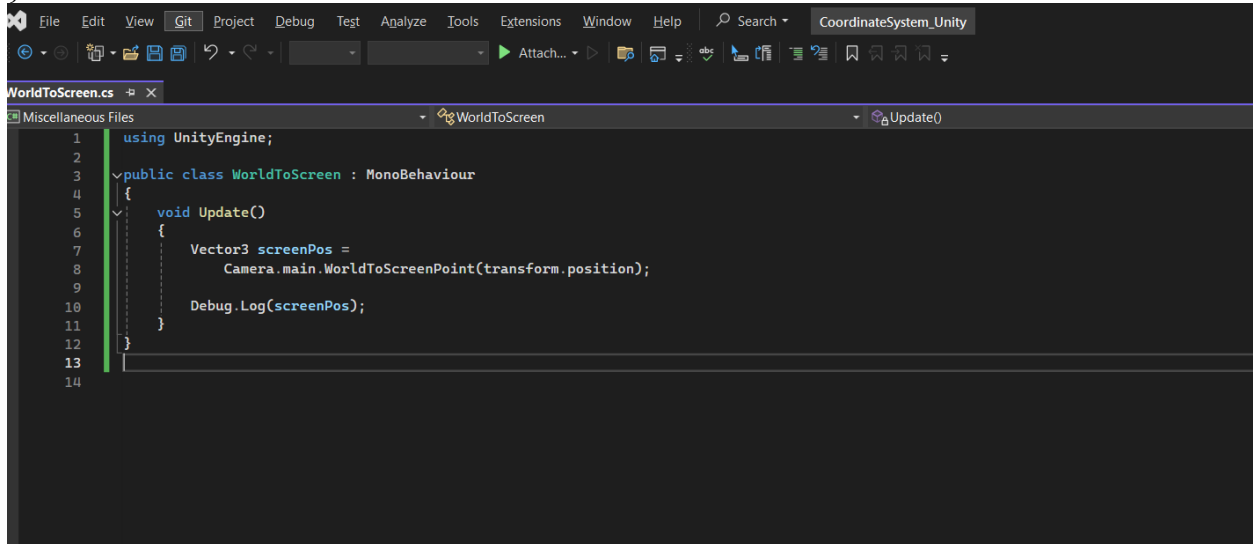
public class WorldToScreen : MonoBehaviour
{

```

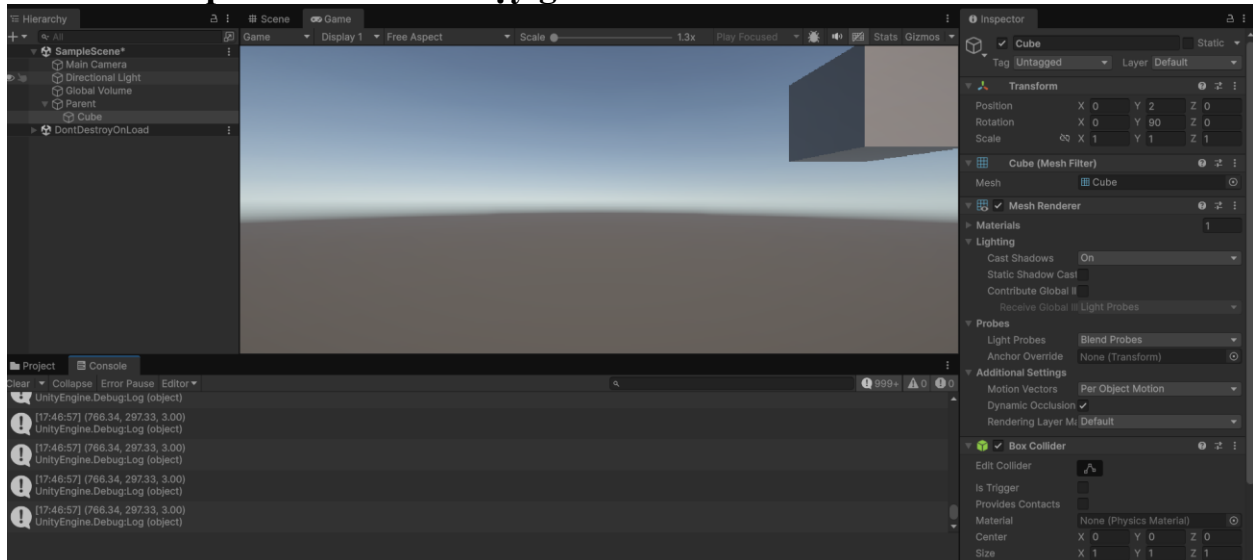
```

void Update()
{
    Vector3 screenPos =
        Camera.main.WorldToScreenPoint(transform.position);
    Debug.Log(screenPos);
}
}

```

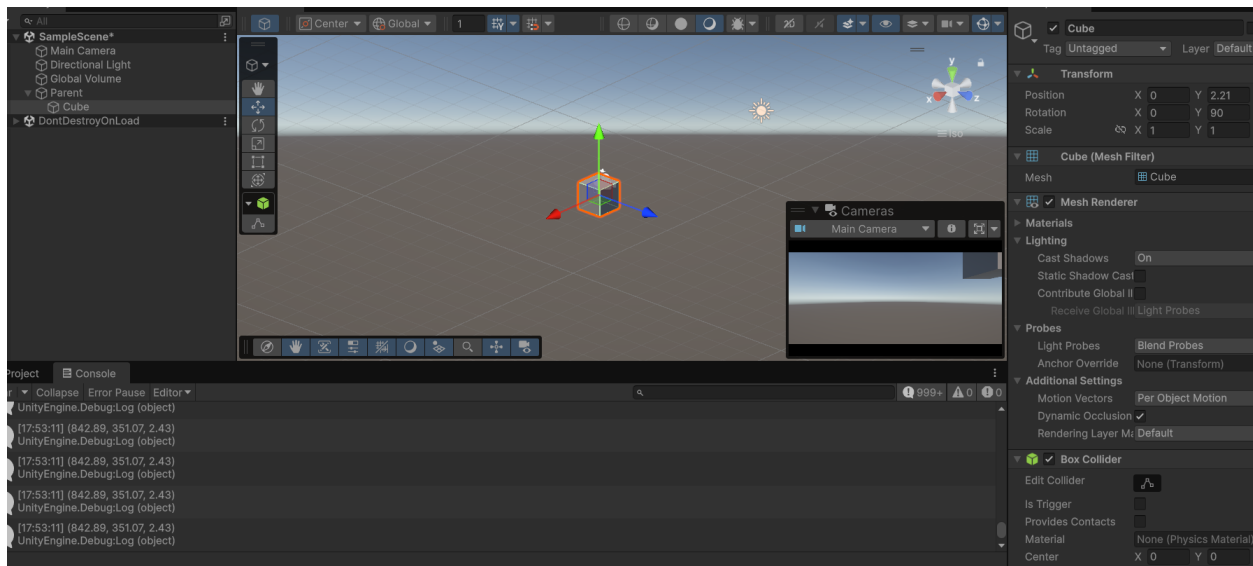


E2. Gắn script vào Cube và chạy game

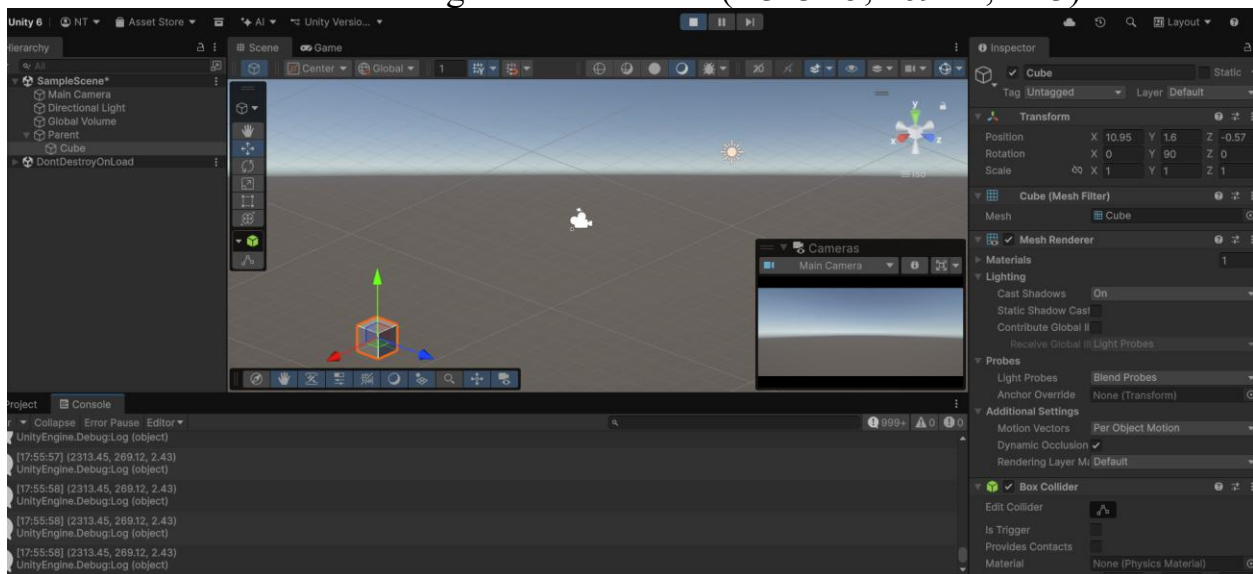


E3. Ghi lại:

- Screen Position khi Cube ở giữa màn hình : (842.89, 351.07, 2.43)



- Screen Position khi Cube ở góc dưới bên trái: (2313.45, 269.12, 2.43)



E4.

_ **Góc tọa độ của Screen Space nằm ở đâu?** Góc dưới bên trái màn hình (0,0)

_ **Screen Space khác World Space như thế nào?** World Space dùng tọa độ 3D, Screen Space dùng pixel 2D

Nhận xét cá nhân

Sau khi làm xong bài tập này, em hiểu rõ hơn cách Unity xử lý không gian 3D và hiển thị lên màn hình 2D. Trước đây khi làm việc với Unity, em thường đặt vị trí hay xoay object theo cảm giác, chưa thực sự hiểu rõ ý nghĩa của các trục tọa độ.

Qua bài này, em đã nắm được cách sử dụng hệ trục x, y, z và biết trục nào dùng để xác định chiều cao, chiều ngang hay chiều sâu của object.

Phần tìm hiểu về Left-Handed Coordinate System giúp em hiểu rõ hơn vì sao object trong Unity lại quay theo một hướng nhất định khi thay đổi Rotation. Nhờ đó, em tránh được việc xoay sai hướng khi thao tác hoặc khi lập trình. Ngoài ra, khi làm việc với Parent và Child, em đã phân biệt rõ được Local Space và World Space, hiểu được sự khác nhau giữa vị trí tương đối và vị trí trong toàn Scene.

Bên cạnh đó, phần Camera giúp em hiểu vì sao cùng một object nhưng khi thay đổi Field of View hoặc vị trí Camera thì kích thước hiển thị lại thay đổi, thậm chí có thể không còn nhìn thấy trên màn hình. Phần chuyển đổi từ World Space sang Screen Space bằng script cũng giúp em hiểu rõ hơn cách Unity xác định vị trí object trên màn hình. Bài tập này giúp em hiểu hơn những kiến thức cơ bản trong Unity giúp tạo nền tảng tốt hơn cho việc học tập của em trên trường và thêm kinh nghiệm cho sau này.